Audio Feature Vectorization Guide

Overview

This document explains the audio feature vectorization system used in the Music Mixing Studio application. The system converts raw audio waveforms into numerical feature vectors that enable similarity search, track recommendations, and intelligent mixing capabilities.

Table of Contents

Introduction

What is Vectorization?

Vectorization is the process of converting audio signals into fixed-size numerical arrays (vectors) that capture essential musical characteristics. These vectors enable:

- **Similarity Search**: Find tracks or segments with similar musical characteristics
- **Recommendations**: Suggest tracks that complement each other
- **Intelligent Mixing**: Automatically find transition points between tracks

Why Two Levels?

The system uses a **hierarchical approach** with two levels of granularity:

1. **Bar-Level Vectors (128 dimensions)**: Capture short-term musical patterns (typically 4 beats)
2. **Phrase-Level Vectors (256 dimensions)**: Capture longer-term musical structure (typically 8-16 bars)

This dual-level approach allows for both precise segment matching and broader musical texture comparison.

System Architecture

```
Audio File (MP3/WAV)
    ↓
[Audio Analysis]
    ├→ BPM Detection
    ├→ Key Detection
    ├→ Waveform Extraction
    └→ Structure Analysis (bars, phrases)
    ↓
[Waveform Data] (80 samples)
    ↓
[Bar-Level Vectorization]
    └→ 128-dim vectors (one per bar)
    ↓
```

```
[Phrase-Level Vectorization]
    └→ 256-dim vectors (one per phrase)
    ↓
[Storage]
    ├→ bar_vecs.json
    ├→ phrase_vecs.json
    ├→ summary.json
    └→ waveform.json
```

Bar-Level Vectorization

Purpose

Bar-level vectors capture the musical characteristics of individual bars (typically 4 beats). These are used for:

- Precise segment matching
- Finding similar rhythmic patterns
- Identifying transition points

Algorithm

Input:

- `waveform` : Array of normalized audio samples (typically 80 values)
- `barCount` : Number of bars in the track

Process:

1. **Segment Division**: Divide waveform into equal segments (one per bar)

   ```
   const barsPerSample = Math.floor(waveform.length / barCount);
   ```

2. **Feature Extraction** (for each bar segment):

   - **RMS (Root Mean Square)**: Measures average energy

     ```
     RMS = √(Σ(sample²) / N)
     ```

   - **Peak**: Maximum amplitude value
   - **Mean**: Average amplitude
   - **Variance**: Measure of amplitude variation

3. **Vector Construction** (128 dimensions):

   ```
   Vector = [
     RMS,           // Feature 0
     Peak,          // Feature 1
     Mean,          // Feature 2
     Variance,      // Feature 3
     ...normalized_samples[0:123]  // Features 4-127
   ]
   ```

4. **Normalization**: Sample values are normalized from [-1, 1] to [0, 1]

   ```
   sample * 0.5 + 0.5
   ```

Code Implementation

```
function generateBarVectors(waveform: number[], barCount: number): number[][] {
  const barsPerSample = Math.floor(waveform.length / barCount);
  const vectors: number[][] = [];

  for (let i = 0; i < barCount; i++) {
    const start = i * barsPerSample;
    const end = Math.min(start + barsPerSample, waveform.length);
    const barSamples = waveform.slice(start, end);

    // Extract statistical features
    const rms = Math.sqrt(
      barSamples.reduce((sum, v) => sum + v * v, 0) / barSamples.length
    );
    const peak = Math.max(...barSamples);
    const mean = barSamples.reduce((sum, v) => sum + v, 0) / barSamples.length;
    const variance =
      barSamples.reduce((sum, v) => sum + (v - mean) ** 2, 0) /
      barSamples.length;

    // Create 128-dim vector
    const vector = [
      rms,
      peak,
      mean,
      variance,
      ...barSamples.slice(0, 124).map((v) => v * 0.5 + 0.5),
    ].slice(0, 128);

    vectors.push(vector);
  }

  return vectors;
}
```

Vector Structure

Each bar vector is a 128-dimensional array:

```
[RMS, Peak, Mean, Variance, Sample₀, Sample₁, ..., Sample₁₂₃]
```

Phrase-Level Vectorization

Purpose

Phrase-level vectors capture longer-term musical structure and texture. These are used for:

- Overall track similarity
- Musical texture matching
- High-level recommendations

Algorithm

Input:

- `barVecs` : Array of bar-level vectors (128-dim each)
- `phraseCount` : Number of phrases in the track

Process:

1. **Grouping**: Group consecutive bars into phrases

   ```
   const barsPerPhrase = Math.floor(barVecs.length / phraseCount);
   ```

2. **Mean Pooling**: Average all bar vectors within each phrase

   ```
   // For each dimension, compute the mean across all bars
   meanVec[dimension] = Σ(barVec[dimension]) / barsPerPhrase
   ```

3. **Derived Features**:

   - **Energy**: Average of first 10 dimensions (captures overall energy)

     ```
     Energy = mean(pooled[0:9])
     ```

   - **Texture**: Measure of variation in dimensions 10-19

     ```
     Texture = mean(|pooled[10:19] - 0.5|)
     ```

4. **Vector Construction** (256 dimensions):

   ```
   Vector = [
     ...pooled[0:127],    // 128 dimensions from mean pooling
     Energy,              // Dimension 128
     Texture,             // Dimension 129
     ...zeros[130:255]    // Padded to 256 dimensions
   ]
   ```

Code Implementation

```
function generatePhraseVectors(
  barVecs: number[][],
  phraseCount: number
): number[][] {
  const barsPerPhrase = Math.floor(barVecs.length / phraseCount);
  const vectors: number[][] = [];

  for (let i = 0; i < phraseCount; i++) {
    const start = i * barsPerPhrase;
    const end = Math.min(start + barsPerPhrase, barVecs.length);
    const phraseBars = barVecs.slice(start, end);

    // Mean pooling: average all bar vectors
    const meanVec = new Array(128).fill(0);
    phraseBars.forEach((bar) => {
      bar.forEach((val, idx) => {
        meanVec[idx] += val;
      });
    });
    const pooled = meanVec.map((val) => val / phraseBars.length);

    // Add derived features
    const energy = pooled.slice(0, 10).reduce((sum, v) => sum + v, 0) / 10;
```

```
    const texture =
      pooled.slice(10, 20).reduce((sum, v) => sum + Math.abs(v - 0.5), 0) / 10;

    // Create 256-dim vector
    const vector = [
      ...pooled,
      energy,
      texture,
      ...new Array(256 - pooled.length - 2).fill(0),
    ].slice(0, 256);

    vectors.push(vector);
  }

  return vectors;
}
```

Vector Structure

Each phrase vector is a 256-dimensional array:

$$[\text{Pooled}_0, \text{Pooled}_1, \ldots, \text{Pooled}_{127}, \text{Energy}, \text{Texture}, 0, 0, \ldots, 0]$$

Feature Extraction Pipeline

Step-by-Step Process

1. **Audio File Input**
   - Accepts MP3, WAV, or other audio formats
   - Extracts metadata (duration, sample rate, bitrate)
2. **Waveform Generation**
   - Converts audio to mono WAV format
   - Extracts 16-bit PCM samples
   - Downsamples to 80 bars using RMS calculation
   - Normalizes to [0, 1] range
3. **Structure Analysis**
   - Estimates BPM (beats per minute)
   - Calculates number of bars: `bars = (duration × BPM) / (60 × 4)`
   - Calculates number of phrases: `phrases = bars / 8` (typical phrase = 8 bars)
4. **Vector Generation**
   - Generates bar-level vectors (128-dim)
   - Generates phrase-level vectors (256-dim)
5. **Storage**
   - Saves vectors as JSON files
   - Stores track summary metadata

Complete Pipeline Code

```
async function analyzeTrack(track_id: string, file_path: string) {
  // 1. Analyze audio file
  const features = await analyzeAudioFile(file_path);

  // 2. Create track summary
  const summary: TrackSummary = {
    tempo_bpm: features.tempo_bpm,
    key: features.key,
    energy: features.energy,
    duration: features.duration,
    phrases: features.phrases,
    bars: features.bars,
  };

  // 3. Generate and save vectors
  await saveFeatures(track_id, summary, features.waveform);

  return summary;
}
```

Vector Storage

File Structure

For each track, vectors are stored in:

```
storage/tracks/{track_id}/features/
├── bar_vecs.json      # Array of 128-dim vectors
├── phrase_vecs.json   # Array of 256-dim vectors
├── summary.json       # Track metadata
└── waveform.json      # Original waveform data
```

JSON Format

bar_vecs.json:

```
[
  [rms, peak, mean, variance, sample₀, ..., sample₁₂₃],
  [rms, peak, mean, variance, sample₀, ..., sample₁₂₃],
  ...
]
```

phrase_vecs.json:

```
[
  [pooled₀, ..., pooled₁₂₇, energy, texture, 0, ..., 0],
  [pooled₀, ..., pooled₁₂₇, energy, texture, 0, ..., 0],
  ...
]
```

Loading Vectors

```
async function loadFeatures(track_id: string) {
  const featuresDir = getFeaturesDir(track_id);

  const barVecs = JSON.parse(
    await fs.readFile(path.join(featuresDir, 'bar_vecs.json'), 'utf-8')
  );
  const phraseVecs = JSON.parse(
    await fs.readFile(path.join(featuresDir, 'phrase_vecs.json'), 'utf-8')
  );
  const summary = JSON.parse(
    await fs.readFile(path.join(featuresDir, 'summary.json'), 'utf-8')
  );

  return { barVecs, phraseVecs, summary };
}
```

Similarity Computation

Cosine Similarity

The system uses **cosine similarity** to compare vectors. This measures the angle between two vectors, making it ideal for high-dimensional feature spaces.

Formula

```
similarity(A, B) = (A · B) / (||A|| × ||B||)
```

Where:

- `A · B` = dot product
- `||A||` = Euclidean norm (magnitude) of vector A
- `||B||` = Euclidean norm (magnitude) of vector B

Implementation

```
function cosineSimilarity(vecA: number[], vecB: number[]): number {
  if (vecA.length !== vecB.length) return 0;

  let dot = 0;
  let normA = 0;
  let normB = 0;

  for (let i = 0; i < vecA.length; i++) {
    dot += vecA[i] * vecB[i];
    normA += vecA[i] * vecA[i];
    normB += vecB[i] * vecB[i];
  }

  if (normA === 0 || normB === 0) return 0;
  return dot / (Math.sqrt(normA) * Math.sqrt(normB));
}
```

Similarity Range

- **1.0**: Identical vectors (same direction)
- **0.0**: Orthogonal vectors (no similarity)
- **-1.0**: Opposite vectors (rare with normalized features)

Use Cases

1. Similar Segment Search

Find segments in other tracks that are similar to a specific position in the current track.

```
async function searchSimilar(
  track_id: string,
  position: number,
  scope: "bar" | "phrase"
) {
  const features = await loadFeatures(track_id);

  // Get reference vector at position
  const refVec = scope === "bar"
    ? features.barVecs[barIndex]
    : features.phraseVecs[phraseIndex];

  // Search across all tracks
  // Compare using cosine similarity
  // Return top K matches
}
```

2. Track Recommendations

Recommend tracks that complement the current track based on overall texture and energy.

```
async function recommendTracks(sourceTrackId: string) {
  const sourceFeatures = await loadFeatures(sourceTrackId);

  // Compare average phrase vectors
  const avgSourcePhrase = mean(sourceFeatures.phraseVecs);

  // Find tracks with similar texture
  // Score based on: texture similarity, key compatibility, energy match
}
```

3. Transition Point Detection

Find optimal points to transition between two tracks.

```
function findBestTransition(
  phrasesA: number[][],
  phrasesB: number[][]
) {
  // Compare all phrase pairs
  // Find highest similarity score
  // Return transition positions
}
```

Code Examples

Example 1: Generating Vectors for a New Track

```
import { analyzeTrack } from './lib/analysis';

// Analyze and vectorize a new track
const summary = await analyzeTrack(
  'track_123',
  '/path/to/audio.mp3'
);

console.log(`Track has ${summary.bars} bars and ${summary.phrases} phrases`);
```

Example 2: Finding Similar Segments

```
import { searchSimilar } from './lib/candidates';

// Find similar bars at position 30 seconds
const similar = await searchSimilar(
  'track_123',
  30.0,  // position in seconds
  'bar', // scope: 'bar' or 'phrase'
  10     // top 10 results
);

similar.forEach(result => {
  console.log(`Track ${result.track_id} at ${result.position}s: ${result.score}`);
});
```

Example 3: Computing Similarity Directly

```
import { loadFeatures, cosineSimilarity } from './lib/analysis';

const featuresA = await loadFeatures('track_123');
const featuresB = await loadFeatures('track_456');

// Compare first bars
const similarity = cosineSimilarity(
  featuresA.barVecs[0],
  featuresB.barVecs[0]
);

console.log(`Bar similarity: ${similarity}`);
```

Technical Details

Vector Dimensions

- **Bar Vectors**: 128 dimensions
    - 4 statistical features (RMS, Peak, Mean, Variance)

    - 124 normalized sample values
- **Phrase Vectors**: 256 dimensions
    - 128 dimensions from mean-pooled bar vectors
    - 1 energy feature
    - 1 texture feature
    - 126 zero-padded dimensions (for future expansion)

Performance Considerations

- **Storage**: JSON format is human-readable but not optimal for large datasets
    - Production recommendation: Use binary formats (`.npz` , `.h5`) or vector databases (Pinecone, Weaviate)
- **Computation**: Cosine similarity is O(n) where n is vector dimension
    - For large libraries, consider approximate nearest neighbor search (ANN)
- **Memory**: Vectors are loaded on-demand
    - Consider caching frequently accessed vectors

Future Enhancements

1. **Deep Learning Embeddings**: Replace hand-crafted features with learned representations
2. **Multi-Scale Features**: Add beat-level and section-level vectors
3. **Temporal Features**: Include time-dependent features (tempo changes, energy curves)
4. **Harmonic Features**: Add chroma features for better key matching

---

Conclusion

The vectorization system provides a robust foundation for music similarity and recommendation features. By capturing both short-term (bar-level) and long-term (phrase-level) musical characteristics, the system enables precise segment matching and high-level track recommendations.

The hierarchical approach balances computational efficiency with musical expressiveness, making it suitable for real-time DJ applications while maintaining the flexibility to incorporate more sophisticated features in the future.

References

- **File Location**: `lib/analysis.ts`
- **Audio Processing**: `lib/audio-analysis.ts`
- **Similarity Search**: `lib/candidates.ts`
- **Recommendations**: `lib/recommendations.ts`

Generated for Music Mixing Studio Application